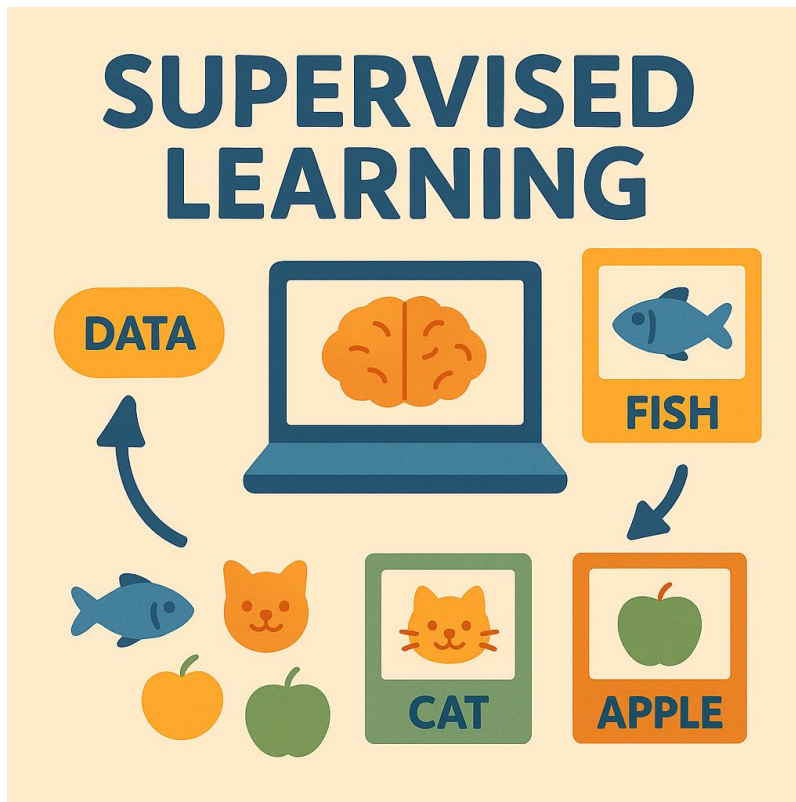


NF2 – MODELS SUPERVISATS



Autor: Generada ChatGPT

Curs d'Especialització en Intel·ligència Artificial i BigData

MP5072 Sistemes d'aprenentatge automàtic

Continguts

Introducció.....	4
Models supervisats de regressió.....	5
Regressió lineal simple.....	6
Avaluar el model de regressió lineal simple	9
MAE (Mean Absolute Error) – Error mig absolut.....	10
MSE (Mean Square Error) – Error quadràtic mig (ECM)	10
RMSE (Root Mean Square Error) – Arrel de l’error quadràtic mig	10
R-quadrat (R^2) <i>R Squared</i>	11
Regressió lineal múltiple	17
Exhaustiu (All-in)	18
Eliminació cap enrere (Backward elimination technique)	18
Selecció endavant (<i>Forward Selection</i>)	21
Eliminació bidireccional (Bidirectional elimination / Stepwise)	23
Tots els models possibles.....	24
Avaluar el model de regressió lineal múltiple.....	25
1.1.1. R^2 ajustat	25
Regressió polinòmica	31
Avantatges d’ús de la regressió polinòmica.....	35
Problemes de la regressió polinòmica	35
Models lineals regularitzats	37
Regularització L1 (<i>Lasso Regression</i>).....	38
Això fa que alguns coeficients es converteixin exactament en zero , fent que el model elimini variables no rellevants.	38
Regularització L2 (<i>Ridge Regression</i>)	40
Per exemple volem predir la demanda d’energia d’una ciutat amb moltes variables (temperatura, hora	

del dia, humitat, dia de la setmana, etc.), on algunes estan correlacionades.....	40
Regularització xarxa elàstica (<i>Elastic Net</i>) – L1 +L2.....	41
Resum	41
Configuració hiperparàmetre	43
Suport Vector Machines	44
Decision Trees i Random Forest.....	45
Adaboost.....	45
Models supervisats de classificació	46
Navie Bayes	46
K-Nearst Neighbors.....	46
Regressió logística.....	46
Suppport Vector Machines	46
Decision Trees i Random Forest.....	46
Adaboost.....	46
REFERÈNCIES	47

Introducció

L'aprenentatge supervisat és un tipus d'aprenentatge automàtic (Machine Learning) en què el model aprèn a partir d'exemples ja etiquetats.

És com un estudiant que aprèn fent exercicis amb les respostes al costat: veu les preguntes (les característiques o *features*) i sap quina és la resposta correcta (l'etiqueta o *target*).

Objectiu: crear un model capaç de predir correctament les etiquetes d'exemples nous, que no ha vist abans.

Formalment l'objectiu és aprendre una funció:

$$f(X) = y$$

On:

- **X** = conjunt de característiques d'entrada (**variables independents**)
- **y** = variable objectiu o etiqueta (**variable dependent**)

El tipus d'aprenentatge podrà ser de regressió o de classificació depenent de la variable independent / objectiu.

Tipus	Tipus de sortida	Exemples
Regressió	Valor numèric continu	Predir el preu d'una casa, la temperatura, la facturació
Classificació	Categoria	Diagnòstic (mallat/sà), predicció de frau (sí/no). Predir la categoria de d'una flor Iris

Models supervisats de regressió

La **regressió** és una de les tasques més fonamentals dins de l'aprenentatge supervisat. El seu objectiu principal és predir un valor numèric continu a partir d'una o més variables d'entrada. A diferència de la classificació, on el model assigna etiquetes discretes com "Sí/No" o "Aprovats/Suspesos", **la regressió busca estimar quantitats**: quant costarà un producte, quina serà la temperatura demà o quin consum energètic tindrà un edifici segons les seves característiques.

Podem entendre la regressió com un procés mitjançant el qual un algorisme aprèn la relació que hi ha entre diverses variables per poder-ne predir una altra. El model intenta construir una funció que representi el vincle entre les dades d'entrada i la sortida esperada. Per exemple, si coneixem la mida, el nombre d'habitacions i la ubicació d'un pis, el model pot mirar d'estimar-ne el preu de mercat. Aquesta capacitat de **capturar relacions quantitatives**.

Començarem aprenent la **Regressió Lineal**, el model més senzill i alhora més fonamental dins d'aquest conjunt de tècniques. Comprendre la regressió lineal és essencial, perquè serveix de base per entendre conceptes més avançats com la relació entre variables, la mesura de l'error o la idea d'ajustar un model a un conjunt de dades. A més, és un model fàcilment interpretable: permet entendre de quina manera cada variable d'entrada influeix en el resultat final.

Ara bé, La regressió lineal no és l'única eina disponible. Molts altres algorismes, inicialment pensats per a classificació, poden adaptar-se per resoldre problemes de regressió amb petites modificacions. És el cas dels **àrbre de decisió (Decision Trees)**, dels **models d'ensamblatge** com **Random Forest** o **Gradient Boosting**, o de les **màquines de vectors suport (Support Vector Machines)**, que en la seva versió **SVR (Support Vector Regression)** permeten treballar amb valors continus.

També existeixen altres mètodes pensats específicament per a regressió, com els **models polinòmics**, que permeten capturar relacions corbades o no lineals; les regressions amb **regularització**, com **Ridge** o **Lasso**, que ajuden a evitar el sobreajustament.

Hi ha tècniques especialitzades per a **sèries temporals**, com **ARIMA** o **Prophet**, especialment útils quan les dades depenen del temps. Encara que aquestes tècniques no s'estudien en profunditat en aquest

mòdul, conèixer-ne l'existència ajuda a triar millor el model més adequat segons el tipus de problema i les característiques de les dades.

El repte de construir un bon model de regressió no consisteix únicament a obtenir prediccions precises. Un model realment útil ha de ser capaç de **generalitzar bé**, és a dir, de funcionar correctament amb dades noves que no ha vist durant l'entrenament. També ha de ser **interpretable**, perquè en entorns professionals sovint és tan important entendre el "per què" el model fa una predicció, com la predicció mateixa. A més, cal que s'adapti al **context d'ús**: no és el mateix predir el preu d'un cotxe de segona mà que el risc financer d'una inversió, i cada cas requereix un enfocament específic.

Podem imaginar la regressió com el procés d'un meteoròleg que vol predir la temperatura de demà. Té dades històriques de dies anteriors —com la humitat, la pressió atmosfèrica o el vent— i intenta descobrir una relació entre aquestes variables i la temperatura final. Si aconsegueix capturar correctament aquest patró, podrà predir amb força precisió el que passarà demà. Aquesta és, en essència, la idea darrere de qualsevol model de regressió: aprendre **com les variables explicatives determinen una resposta numèrica**.

En resum, la regressió és una de les pedres angulars del *Machine Learning* perquè permet predir valors continus, analitzar relacions entre variables i comprendre millor el comportament de sistemes complexos. Dominar-ne el funcionament és un pas imprescindible per desenvolupar models predictius fiables, capaços de generalitzar i de donar suport a la presa de decisions en entorns reals.

Regressió lineal simple

La **regressió lineal simple** és, sens dubte, el model de regressió més senzill i alhora més fonamental dins del *Machine Learning*. La seva missió és representar la relació que hi ha entre una variable d'entrada (o predictor) i una variable de sortida (o objectiu) mitjançant una **línia recta**.

Podem pensar-hi com intentar ajustar una recta que "passi pel mig" dels punts d'un núvol de dades. Aquesta recta descriu la tendència general: quan la variable X augmenta, com es comporta Y? Creix també? Disminueix? Es manté estable?

El model matemàtic és molt senzill i respon a una equació de primer grau:

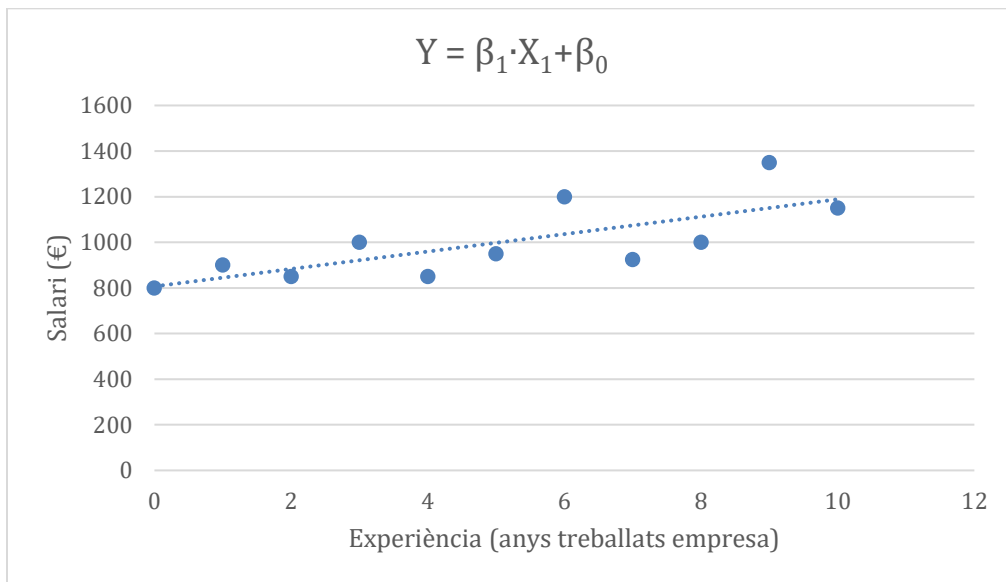
$$Y = \beta_1 \cdot X_1 + \beta_0$$

Y = variable dependent (VD). Per exemple el sou dels empleats.

X_1 = variable independent (VI). Aquesta variable farà variar la variable dependent. Per exemple els anys que el treballador porti treballant a l'empresa.

β_1 = És la pendent de la recta o el coeficient. Indica quan s'ha d'incrementar el sou per cada any que el treballador està a l'empresa.

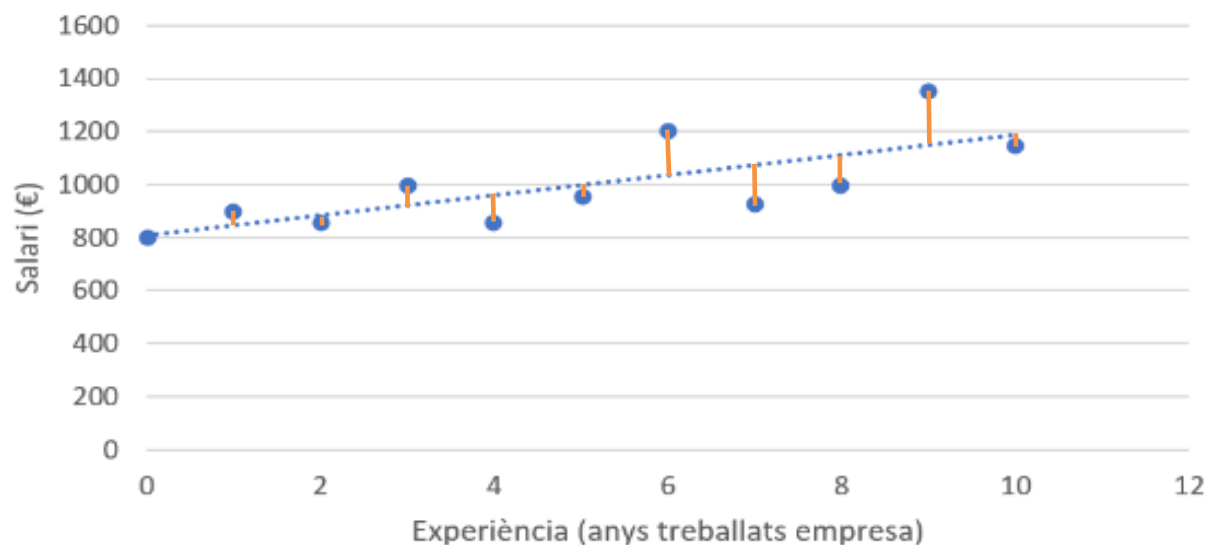
β_0 = Constant. El valor inicial que agafarà la variable dependent (VD) si no existís la variable independent (VI).



$$\text{Salari} = Y = \beta_0 + \beta_1 \cdot \text{Experiència}$$

Si ens hi fixem en la β_1 (pendent/coeficient) veiem que ens indica per cada any quin és l'increment de salari.

Per trobar la recta podem utilitzar el **mètode del mínims quadrats (Least Squares Regression)**



Els punts blaus representen les observacions reals que tenim de les dades i les línies verticals de color verd que van de les observacions reals fins a la línia de regressió són la diferència del que diu el nostre model i la dada real.



En aquest punt veiem que hi ha una diferència de la dada real Y_i i la dada estimada del nostre model $\hat{Y}_i$. El que hem de mirar és que la longitud de la línia verda entre Y_i (valor real) i $\hat{Y}_i$ (valor estimat) sigui el més petit possible. Això és el que fa precisament el mètode dels mínims quadrats.

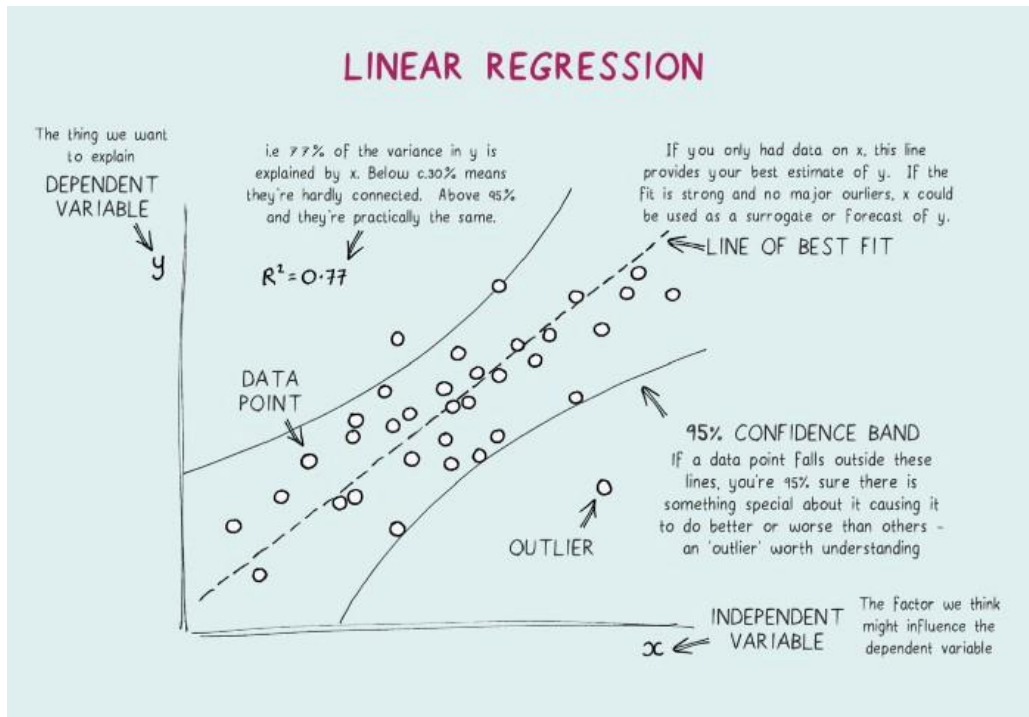
$$\min \sum_i (y_i - \hat{y}_i)^2$$

Sumar totes les diferències. Les elevem al quadrat perquè tots siguin positives i llavors les sumem. Aquestes sumes de diferències el quadrat. De totes les rectes ens quedarem en aquella que aquest valor sigui el més petit.

Minimitza els quadrats de les diferències entre la dada real i la predicció.

El model trobarà el coeficient β_1 i la constant la β_0

Altres mètodes podrien ser **el mètode de màxima versemblança (Maximum Likelihood Estimation) o mitjançant el descendent del gradient.**



Il·lustració 1 By Dan White 1000 from Shutterstock

Avaluar el model de regressió lineal simple

Encara que hem ajustat un model és necessari verificar la seva eficàcia, ja que encara que la línia que millor s'ajusti a les observacions entre totes les possibles, el model pot ser dolent.

Les mètriques les utilitzarem per avaluar els models i veure si aquests són bons o generalitzaren bé.

Existeixen multitud de mètriques, però depenent del tipus de problema utilitzarem unes o unes altres.

	Edats		
	Valor real (y)	Valor predit ($\hat{y}$)	Diferència valor absolut
	32	35	3
	23	28	5
	85	95	10
Mitjana	46,66667	52,66667	6

Considerem la diferència entre un valor predit i un de real com l'error, però com que els valors "reals" es basen en observacions (que poden tenir variància aleatòria) direm que la diferència és entre valor

predit ($\hat{y}$) i un valor observat (y). En aquest cas en termes matemàtics i/o estadístics la diferència s'anomena **residual**.
Podem resumir els residuals de totes les prediccions per calcular la **pèrdua** general del model com a mesura del seu rendiment predictiu

MAE (Mean Absolute Error) – Error mig absolut

També conegut com a funció de pèrdua L1.

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$
$$MAE = \frac{1}{3} (|32 - 35| + |23 - 28| + |85 - 95|) = \frac{1}{3} (3 + 5 + 10) = 6$$

MSE (Mean Square Error) – Error quadràtic mig (ECM)

També conegut com a funció de pèrdua L2.

Penalitzta els errors grans ja que elevem el quadrat abans de fer la mitjana.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
$$MSE = \frac{1}{3} ((32 - 35)^2 + (23 - 28)^2 + (85 - 95)^2) = \frac{1}{3} (3^2 + 5^2 + 10^2) = 44.666$$

És bó el resultat de 44.66?

És difícil de saber perquè el valor no s'expressa en una unitat de mesura significativa. Sabem que quan menor sigui el valor, millor serà el nostre model. Aquest valor és útil per comparar dos models i trobar quin té millor rendiment, però no el podem sospesar massa perquè el valor no s'expressa en una unitat de mesura significativa.

RMSE (Root Mean Square Error) – Arrel de l'error quadràtic mig

Igual que l'anterior, però la diferència entre aquest (RMSE) i l'anterior (MSE) és que fem l'arrel quadrada al final.

Aquesta mètrica és útil per expressar la pèrdua amb les mateixes unitats que el propi valor de la predicció. En aquest cas les edats

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$
$$RMSE = \sqrt{\frac{1}{3} (3^2 + 5^2 + 10^2)} = 6.683$$

Si valorem el valor RMSE (6.683) i el de MAE (6) veiem que el primer empitjorem perquè la última diferència (10) l'elevem al quadrat per donar-li més importància.

Compte amb els valors de MAE i RMSE perquè moltes vegades quan prediem resultats esbiaixats l'error també tindrà aquest biaix. Si tenim un error amb molt biaix la mitjana ens pot dir poca cosa.

R-quadrat (R^2) *R Squared*.

R-quadrat és una de les mesures més utilitzades per mesurar la qualitat de l'ajust. El coeficient R^2 és la **correlació** entre els valors reals i les prediccions. Això tindrà un valor entre 0 i 1. El que volem és aconseguir tenir un valor més pròxim a 1 possible.

Quan més gran sigui la R-quadrat (R^2) més bó serà el nostre model ja que tots els punts cauen a dins de la nostra recta de regressió.

En una regressió lineal el que fem és buscar el mínim de les distàncies entre els valors predits i els reals:

$$\text{minimitzar} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Suma dels quadrats dels residus

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Suma dels quadrats totals

Aquest factor no depèn del model sinó que simplement avalua les diferències entre les coordenades y i la mitjana de cadascun d'ells

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Veien a la fórmula que R^2 tendirà a 1 quan SS_{res} (la suma dels quadrats dels residus) sigui més petit o vagi cap al 0 (zero).

Una altra interpretació seria dir com de bo és el nostre model si agaféssim SS_{tot} (suma dels quadrats totals) com a recta de regressió.

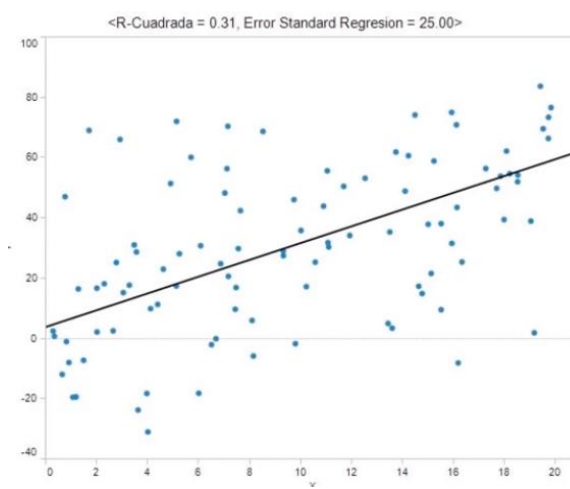
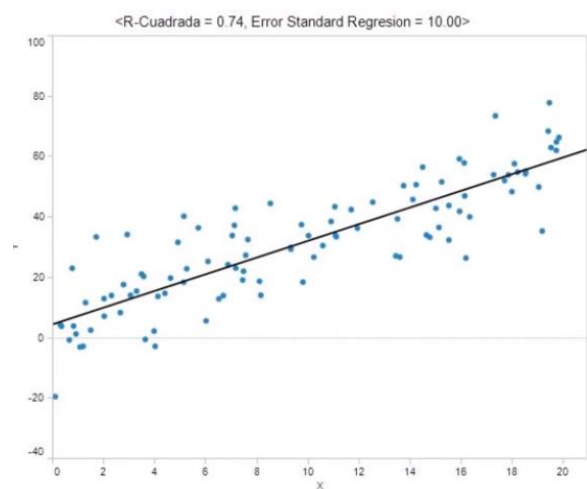
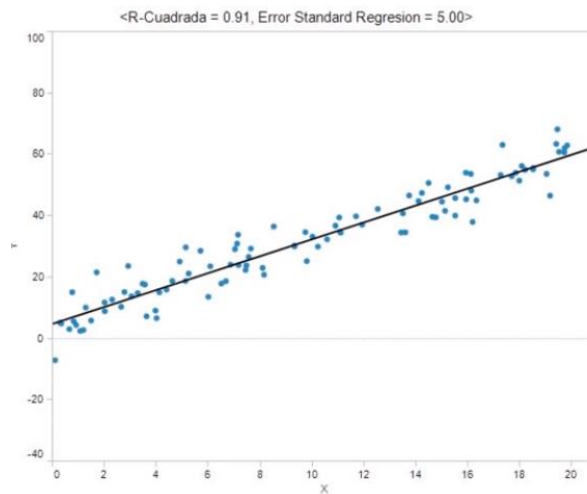
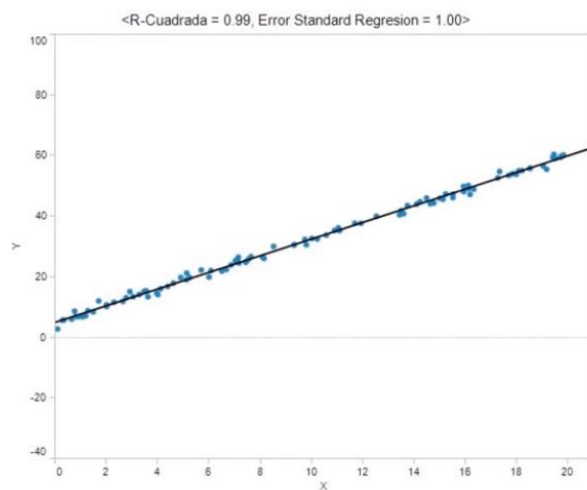
R^2 podria ser negatiu? Si, però això significaria que el model trobat és pitjor que si agafem la mitjana com a valor de predicció.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

En l'exemple:

$$R^2 = 1 - \frac{((32 - 35)^2 + (23 - 8)^2 + (85 - 95)^2)}{((32 - 46.67)^2 + (23 - 46.67)^2 + (85 - 46.67)^2)} = 1 - \frac{134}{2.244,667} = 0,940$$

Fixem-nos en els següents gràfics.



Overfitting en la Regressió

Per veure si ens trobem davant d'un cas o no d'overfitting en un problema de regressió és mirant R^2 .

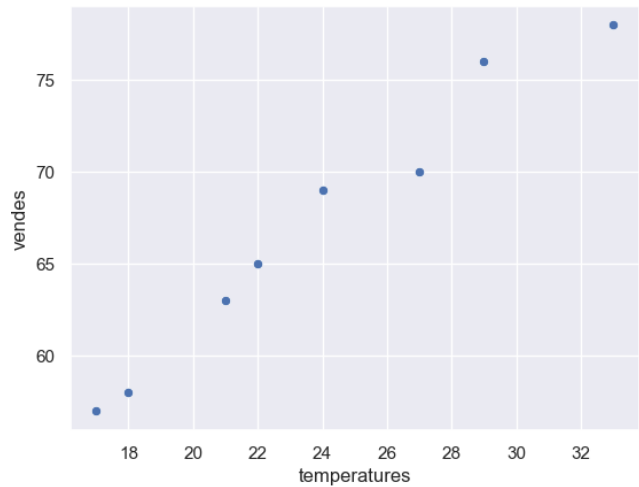
Cas	R^2 train	R^2 test	Comentari
Overfitting	0.98	0.65	Molta diferència entre <i>train/test</i>
Bona generalització	0.70	0.68	Poca diferència i <i>train</i> millor.
Underfitting	0.50	0.45	Poca diferència, però valor R^2 baix.
Regularització o aleatorietat	0.75	0.80	Poca diferència, però <i>test</i> millor

Exemple – Venda de gelats

Un establiment vol predir la venda de gelats que tindrà en un dia determinat per poder aprovisionar-se adequadament. [Enllaç GitHub](#)



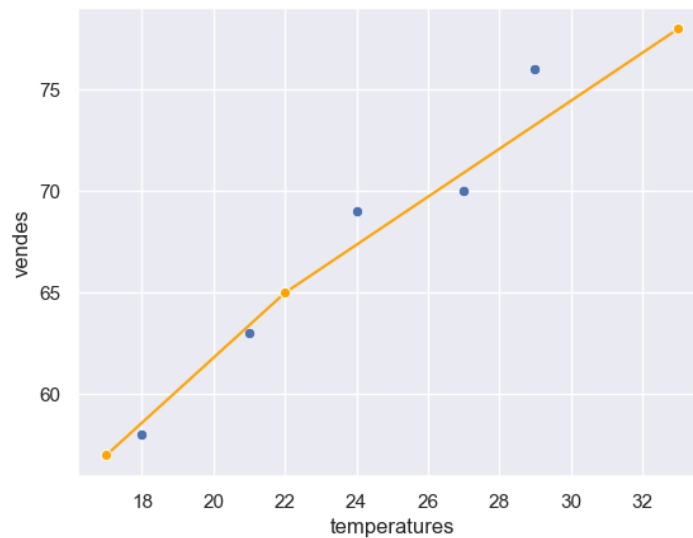
	Temperatura	Vendes
ENTRENAMENT	18	58
	21	63
	24	69
	27	70
	29	76
VALIDACIÓ	33	78
	17	57
	22	65



Veiem que existeix una suposada relació lineal entre x (temperatura) i y (vendes).

Si el model obtingut mitjançant l'entrenament és el següent $Y = 1.52538X + 30.89594$

Podem veure la comparació dels valor de y i $\hat{y}$

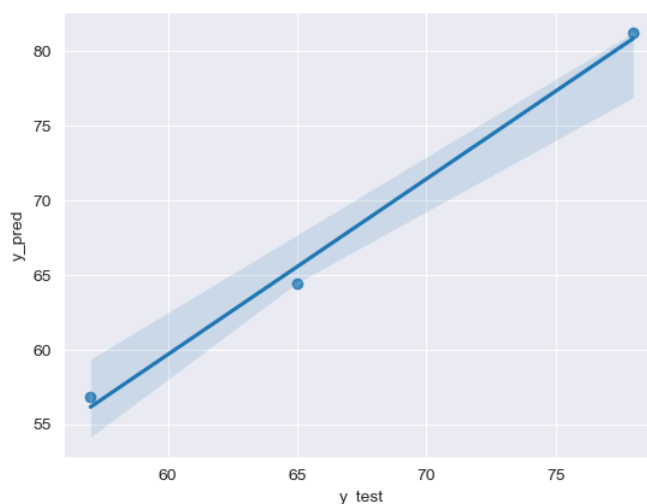


Temperatura x	Vendes y	Vendes $\hat{y}$	$ y - \hat{y} $	$(y - \hat{y})^2$	$(y - \bar{y})^2$
18	58				
21	63				
24	69				
27	70				
29	76				
18	58	81.2334	3.2334	10.4553	128
21	63	56.8274	0.1726	0.0297	
24	69	64.4543	0.5457	0.2977	
SUMA	200		3.95178	10.7829	
MITJANA	66.6667		1.31726	3.5944	

$$MSE = \frac{1}{n} \sum_{i=1}^n (\gamma_i - \hat{\gamma}_i)^2 = 3.5944$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\gamma_i - \hat{\gamma}_i)^2} = \sqrt{MSE} = \sqrt{3.5944} = 1.8971$$

$$R^2 = 1 - \frac{\sum (\gamma_i - \hat{\gamma}_i)^2}{\sum (\gamma_i - \bar{\gamma})^2} = 1 - \frac{10.7829}{224.66667} = 0,95200$$



Veiem amb l'avaluació del model que tenim un coeficient de determinació R^2 força bó amb un valor de **0,952**.

Scatter plot de y_{pred} i y_{test} sembla que també compleix una recta a on no hi ha massa error.

Exercici Beisbol

Volem analitzar si existeix una relació entre el número de batejos (que realitza un equip de beisbol i el número d'entrades que aconseguix.

En cas d'existir aquesta relació caldria establir un model per poder predir el resultat d'un partit.

Regressió lineal múltiple

En una regressió lineal múltiple també tenim una variable depenent (VD) i aquesta la prediem a través de moltes altres variables independents (VI).

En la regressió lineal múltiple no ens conformem amb una sola variable explicativa, sinó que volem entendre com diversos factors junts influeixen sobre una mateixa variable objectiu. És el cas real més habitual: les vendes d'una empresa depenen del preu, de la publicitat, de la temporada, de la competència...

Per exemple, la nota d'un estudiant no només ve determinada per les hores d'estudi, sinó que pot venir influenciada per altres variables: hores de descans, hores que ha assistit a classe,... Cadascuna d'aquestes variables tindrà un coeficient ($a_1, a_2, \dots, a_n$).

$$Y = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \beta_0$$

El primer que hem de tenir en compte és que no totes les variables independents seran útils o ens aportaran informació al model.

$$Y = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \beta_0$$

No és bó quedar-nos amb totes les variables perquè a vegades no per tenir més variables tenim més informació o el model serà millor, sinó que a vegades tenim un model que prediu el mateix en poques variables que amb moltes.

Un altre problema afegit és que quan haguem de defensar el model si hi ha moltes variables costarà molt defensar-lo i explicar com funciona.

Com podem fer per seleccionar les variables correctes? Doncs hi ha diferents mètodes:

Exhaustiu (*All-in*)

Eliminació cap enrere (*Backward elimination technique*)

Selecció endavant (*Forward Selection*)

Eliminació bidireccional (*Bidirectional elimination*)

Comparació de puntuacions (*scores*)

Quina de les tècniques anteriors és la millor? Doncs depèn. No totes donaran el mateix model. Per tant recordem que el nostre objectiu alhora de construir un model de regressió és reduir el màxim l'error de quadrats (SSE) sense caure en **overfitting**.

Hem de poder crear un model en menys variables possibles per definir la variable depenent.

Si afegim més variables independents creem més relacions entre ells. Per tant podem caure a la **multicol·linealitat**. Per tant, el cas ideal és que totes les variables independents estiguin correlacionades amb la variable depenent, però no entre elles.

Exhaustiu (All-in)

És quan posem totes les variables en el model, però ja hem dit que no és una bona pràctica.

L'utilitzarem si tenim coneixement previ i ja sabem que totes les variables han d'estar a dins del model.

Això és perquè tenim algun expert que sap que s'han d'utilitzar totes.

Un altra opció per introduir totes les variables al model és quan ens ho exigeixen.

Preparació prèvia per realitzar el mètode de l'eliminació cap enrere (*backward elimination*).

Eliminació cap enrere (Backward elimination technique)

La tècnica d'eliminació cap enrere és quan agafem totes les variables independents per generar el model i anem eliminant-ne un a una mentre hi hagi alguna variable amb un p-valor > a un cert un nivell de significació α .



Passos:

- **PAS 1:** Establim el nivell de significació (SL/α) per estar dins el model ($\alpha \leq 0.05$)
- **PAS 2:** Calculem el model amb totes les possibles variables independents
- **PAS 3:** Agafem la variable independent amb el p-valor més gran
 - Si p-valor > α llavors passem al PAS 4
 - Altrament passem al PAS 5 - Fi
- **PAS 4:** Eliminem la variable predictora que té el p-valor més gran i tornem a calcular el model
- **PAS 5:** Fi

Exemple:

Tenim les variables X_1, X_2, X_3, X_4, X_5 ,

PAS 1: Establim el nivell de significació a 0.05

PAS 2: Calculem el model amb totes les possibles variables independents.

$$Y = \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \beta_5 X_5 + \beta_0$$

PAS 3: Comprovem el p-valor de totes les variables independents i agafem la més alta i la comparem amb α

- **p-valor_{x1} = 0.0**
- **p-valor_{x2} = 0.691**
- **p-valor_{x3} = 0.155**
- **p-valor_{x4} = 0.943**
- **p-valor_{x5} = 0.0**

El p-valor de X_4 (0.943) és superior al de α (0.05). Per tant traiem anem al PAS 4.

PAS 4: Eliminem la variable X_4 del model i anem al PAS 2

PAS 2: Calculem el model

$$Y = \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_5 X_5 + \beta_0$$

PAS 3: Comprovem el p-valor de totes les variables independents i agafem la més alta i la comparem amb α

- **p-valor_{x1} = 0.0**
- **p-valor_{x2} = 0.674**
- **p-valor_{x3} = 0.155**
- **p-valor_{x5} = 0.0**

El p-valor de X_2 (0.674) és superior al de α (0.05). Per tant traiem anem al PAS 4.

PAS 4: Eliminem la variable X_2 del model i anem al PAS 2

PAS 2: Calculem el model

$$Y = \beta_1 X_1 + \beta_3 X_3 + \beta_5 X_5 + \beta_0$$

PAS 3: Comprovem el p-valor de totes les variables independents i agafem la més alta i la comparem amb α

- **p-valor_{x1} = 0.0**
- **p-valor_{x3} = 0.04**
- **p-valor_{x5} = 0.0**

El p-valor de X3 (0.04) és inferior al de α (0.05). Per tant, ja estem i anem al PAS 5

PAS 5: Ens quedem amb el següents model

$$Y = \beta_1 X_1 + \beta_3 X_3 + \beta_5 X_5 + \beta_0$$

Vídeos:

- [Statistics 101: Multiple Regression, Backward Elimination](#), *Brandon Foltz*

Selecció endavant (*Forward Selection*)

Aquest mètode no és l'efecte contrari o els passos del mètode anterior invertits.



Passos:

- **PAS 1:** Establir el nivell de significació (SL/α) per entrar a dins el model ($\alpha = 0.05$)
- **PAS 2:** Calcular tots els models de regressió amb cadascuna de les variables independents i ens quedarem amb el model que tingui el **p-valor més petit**.
- **PAS 3:** Conservarem la variable independent del pas anterior i ajustem tots els possibles models incloent aquesta variable independent
- **PAS 4:** Considerem la variable independent amb el menor p-valor.
Si $p\text{-valor} < \alpha$ llavors passem al PAS 3ç
Altrament passem al PAS 5 - Fi.
- **PAS 5:** Fi

Exemple:

Tenim les variables X_1, X_2, X_3, X_4, X_5 ,

PAS 1: Establir el nivell de significació a 0.05

PAS 2: Calculem tots els models amb una sola variable independent.

$$Y_1 = \beta_1 X_1 + \beta_0$$

$$Y_2 = \beta_2 X_2 + \beta_0$$

$$Y_3 = \beta_3 X_3 + \beta_0$$

$$Y_4 = \beta_4 X_4 + \beta_0$$

$$Y_5 = \beta_5 X_5 + \beta_0$$

El model que té el menor p-valor és Y_2 amb un valor de 0.0097

PAS 3: Conservem X_2 i ajustem tots els models incloent les altres variables independents.

$$Y_{21} = \beta_1 X_1 + \beta_2 X_2 + \beta_0$$

$$Y_{23} = \beta_3 X_3 + \beta_2 X_2 + \beta_0$$

$$Y_{24} = \beta_4 X_4 + \beta_2 X_2 + \beta_0$$

$$Y_{25} = \beta_5 X_5 + \beta_2 X_2 + \beta_0$$

El model que té el menor p-valor és Y_{25} amb un valor de 0.035

PAS 4: Ens quedem amb el model de Y_{25} i recalculem altre cop tots els models amb totes les variables

$$Y_{251} = \beta_1 X_1 + \beta_2 X_2 + \beta_5 X_5 + \beta_0$$

$$Y_{253} = \beta_3 X_3 + \beta_2 X_2 + \beta_5 X_5 + \beta_0$$

$$Y_{254} = \beta_4 X_4 + \beta_2 X_2 + \beta_5 X_5 + \beta_0$$

El model que té el menor p-valor és Y_{254} amb un valor de 0.073

Com que el p-valor $> \alpha$ Finalitzem i acabem el procés agafant el model Y_{25}

Una altra variant d'aquesta tècnica és utilitzar el valor de R^2 en el PAS2 per quedar-nos amb el model que millor s'ajusti. Amb un llindar de R^2 especificat, per exemple 0.95.

<https://www.youtube.com/watch?v=ckojLIMPtLw>

Vídeos:

[Statistics 101: Multiple Regression, Forward Selection](#), *Brandon Foltz*

Eliminació bidireccional (Bidirectional elimination / Stepwise)



Aquest mètode intenta utilitzar la bondat dels dos anteriors.

- **PAS 1:** Establirem dos nivells de significació. Aquell per entrar al model i aquell per sortir-ne (SLIN/ α_{in} i SLSTAY/ α_{stay}) ($\alpha_{in} = \alpha_{stay} = 0.05$)
- **PAS 2:** Utilitzarem el *Forward Selection* amb les variables noves amb p-valor $< SLIN/\alpha_{in}$
- **PAS 3:** Utilitzarem TOTS els passos del *Backward elimination technique* per mantenir les variables antigues p-valor $< SLSTAY/\alpha_{stay}$
- Repetirem el pas 2 i pas 3 fins que no hi hagi variable per entrar o per sortir.
 - Afegim la millor variable i eliminem les que sobren.
 - Cada vegada que afegim una variable ens assegurem que les variables que es queden són les millors.
- **PAS 4:** No hi ha variables per entrar o per sortir.

Vídeos:

- [Statistics 101: Multiple Regression, Stepwise Selection](#), Brandon Foltz

Tots els models possibles

Aquest mètode també anomenat per força bruta el que fa és calcular tots els models possibles i escollir el que millor s'ajusti.

- **PAS 1:** Seleccionem un criteri alhora d'escollir un model
- **PAS 2:** Generar tots els models possibles de regressió. Si tenim N variables tindrem 2^{N-1} combinacions/models possibles.
- **PAS 3:** Seleccionarem el model amb el millor criteri escollit.

Per exemple amb 10 columnes hauríem de generar 1.023 models!!!!

Avaluar el model de regressió lineal múltiple

1.1.1. R^2 ajustat

Ja hem vist R^2 . En el cas de regressió lineal múltiple tenim un problema i és que R^2 sempre quedarà igual. O sigui no empitjorà cada vegada que afegim variables.

Com que la suma dels quadrats reals no canvia al fet d'afegir variables, R^2 sempre s'incrementarà o com a mínim no empitjorà. Per tant, si només mirem R^2 podem caure en l'error d'afegir variables al model i “sobre inflar-lo” inútilment

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = R^2 = 1 - \frac{\sum(\gamma_i - \hat{\gamma}_i)^2}{\sum(\gamma_i - \bar{\gamma})^2}$$

Per això en el cas de regressions lineals múltiples ens ajudem del **R^2 ajustat**.

Aquest indicador no només es basa en el nombre de variables del model sinó que busca l'equilibri entre les variables independents i el nombre d'observacions.

L' R^2 ajustat pot arribar a ser negatiu si tenim **moltes variables independents i poques observacions**.

En termes estadístics afegir un variable estem perdent un grau de llibertat, i aquesta pèrdua comporta un cost.

Els graus de llibertat són: $DFE = n - p - 1$

A mesura que anem afegint variables

R^2 ajustat té diferents fórmules per calcular-lo on:

- n : És el número d'observacions
- p : És el número de variables independents

$$R_a^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

$$R_a^2 = 1 - \left(\frac{n - 1}{SS_{tot}} \right) \cdot MSE$$

$$R_a^2 = R^2 - \frac{p(1 - R^2)}{n - p - 1}$$

Exemple 1 (augmentem el número de variables sense augmentar observacions): En el següent exemple veiem que augmentant el nombre de variables de 3 a 100 empitjorem R^2 ajustat



$$R_a^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

$p = 3, n = 100, R^2 = 0.7326$	$p = 19, n = 100, R^2 = 0.7651$
$R_a^2 = 1 - (0.2674) \frac{99}{96}$	$R_a^2 = 1 - (0.2349) \frac{99}{80}$
$R_a^2 = 1 - 0.2758$	$R_a^2 = 1 - 0.2907$
$R_a^2 = 0.7242$	$R_a^2 = 0.7093$

Exemple 2 (augmentem el número de variables sense augmentar observacions) : En aquest segon exemple veiem que l'empitjorament ha estat major que l'anterior, perquè el número d'observacions és més petit.



$$R_a^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

$p = 3, n = 50, R^2 = 0.7326$	$p = 19, n = 50, R^2 = 0.7651$
$R_a^2 = 1 - (0.2674) \frac{49}{46}$	$R_a^2 = 1 - (0.2349) \frac{49}{30}$
$R_a^2 = 1 - 0.5755$	$R_a^2 = 1 - (0.3837)$
$R_a^2 = 0.7152$	$R_a^2 = 0.6163$

Resum :



HYPOTHETICAL

$$p = 3, n = 100, R^2 = 0.7326 \quad R_a^2 = 0.7242$$

$$p = 19, n = 100, R^2 = 0.7651 \quad R_a^2 = 0.7093$$

$$p = 3, n = 50, R^2 = 0.7326 \quad R_a^2 = 0.7152$$

$$p = 19, n = 50, R^2 = 0.7651 \quad R_a^2 = 0.6163$$

Això ens demostra que sempre hem de mantenir el model tan simple com sigui possible

Mira també els exemples de *Backward* i *Fordward* com evoluciona R^2 i R^2 ajustada

Explicació de la sortida del *summary* de **StatsModels** de Python

- **No. Observacions:** N° d'observacions
- **Df Residuals:** Els grau de llibertat: observacions - variables independents – 1
- **Df Model:** El número de variables independents (excloent el terme independent)

```
lr_ols.summary()
```

OLS Regression Results

Dep. Variable:	Profit	R-squared:	0.952
Model:	OLS	Adj. R-squared:	0.943
Method:	Least Squares	F-statistic:	113.9
Date:	Sun, 04 Dec 2022	Prob (F-statistic):	3.78e-18
Time:	22:23:14	Log-Likelihood:	-370.11
No. Observations:	35	AIC:	752.2
Df Residuals:	29	BIC:	761.5
Df Model:	5		
Covariance Type:	nonrobust		

Càlcul de R^2

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

```
# Calculem els residus dels valors predits amb els reals
residus = y - y_pred

# Calculem la suma dels quadrats dels residus
SSR = np.dot(residus,residus)
print("Suma dels quadrats dels residus (SSR):",SSR)

# Calculam la suma dels quadrats totals
centre_y = y - y.mean()
SST = np.dot(centre_y,centre_y)
print("Suma dels quadrats totals (SST):",SST)

#Calculem R2
R_quadrat = 1 - (SSR / SST)
print("R-quadrat:",R_quadrat)
```

Càlcul de R^2 ajustat

$$R_a^2 = 1 - \frac{(1 - R^2) (n - 1)}{n - p - 1}$$

```
#Graus de llibertat
df_resid = model.df_resid

# Número d'observacions
nobs = df_med.shape[0]

#Calculem R2 ajustat
R_quadrat_ajust = 1 - (nobs-1/df_resid * (1-R_quadrat))
print("R-quadrat ajustat:",R_quadrat_ajust)
```

Alerta

Regressió polinòmica

Fins ara hem vista dos tipus de regressions lineals:

- Regressió lineal simple: Conté una sola variable independent

$$Y = \beta_1 X_1 + \beta_0$$

- Regressió lineal múltiple: Conté més d'una variable independent

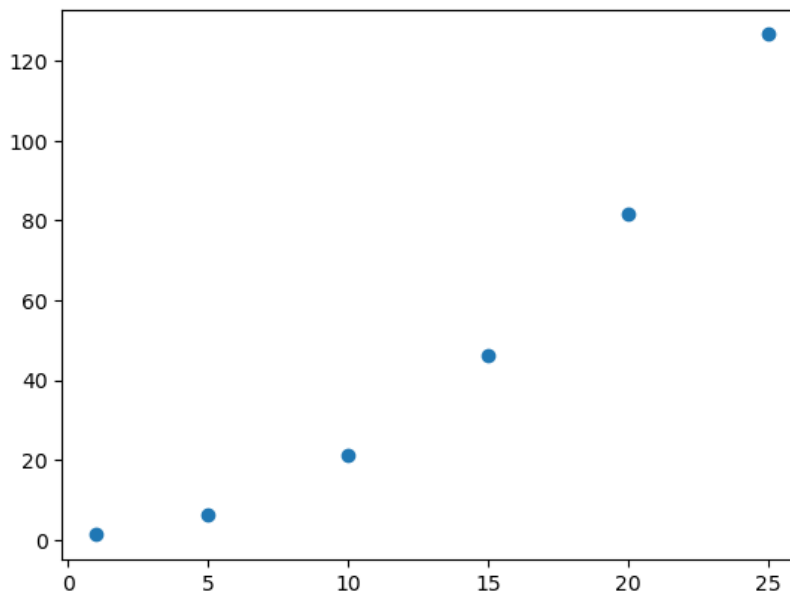
$$Y = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \beta_0$$

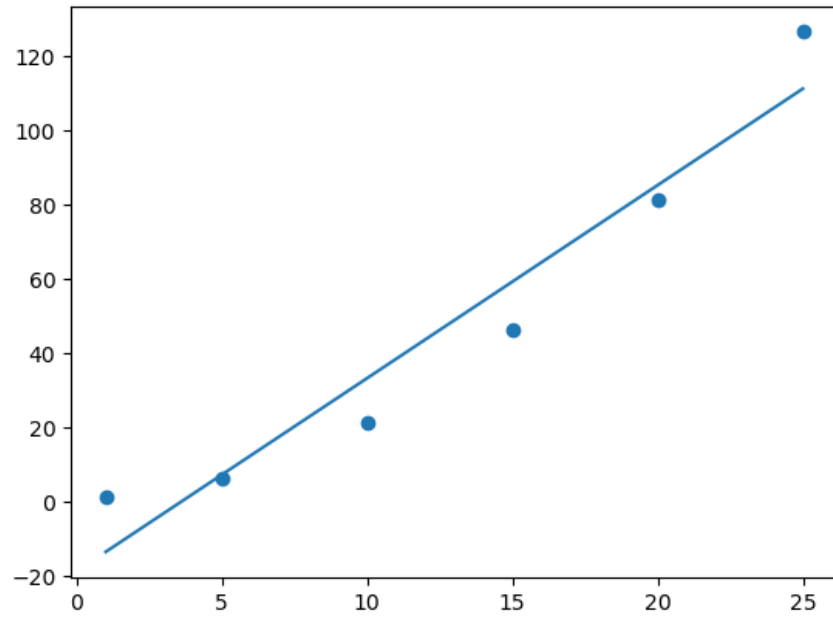
En la regressió polinòmica tenim una sola variable independent, però aquesta expressada en un polinomi de grau n , on aquesta n sigui l'exponent (potència) més elevat de la variable independent.

$$Y = \beta_n X_1^n + \dots + \beta_2 X_1^2 + \beta_1 X_1 + \beta_0$$

Exemple 1

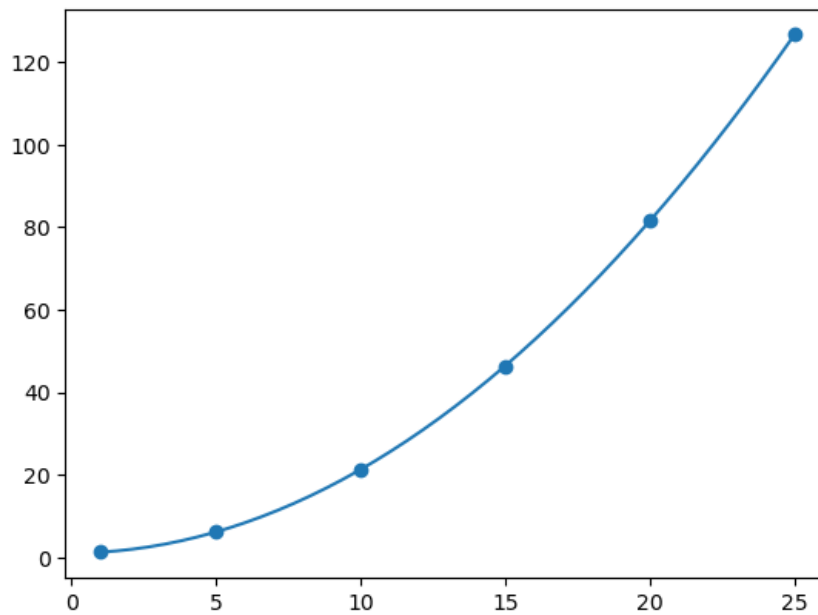
Amb aquest conjunt de dades podem intuir que potser un model que s'ajusta a les nostres dades seria un model de regressió lineal simple.





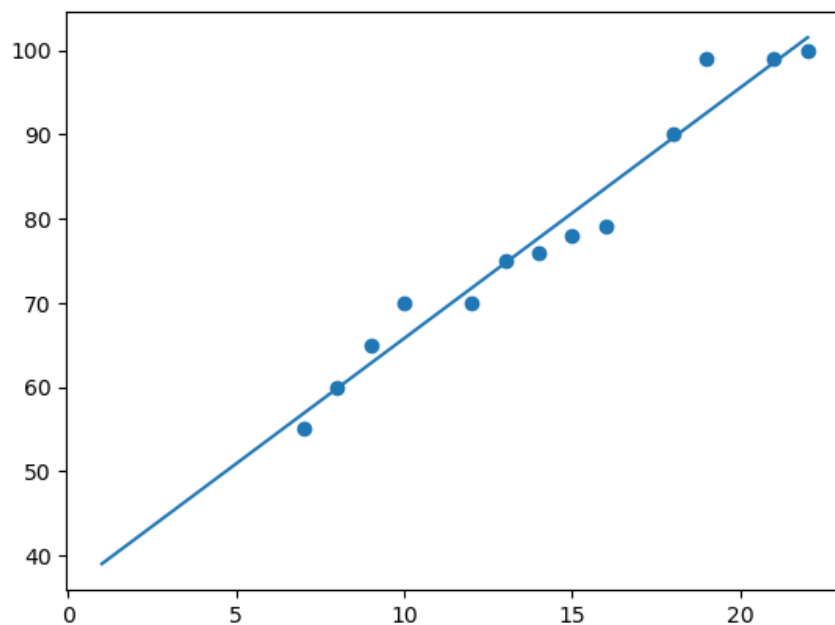
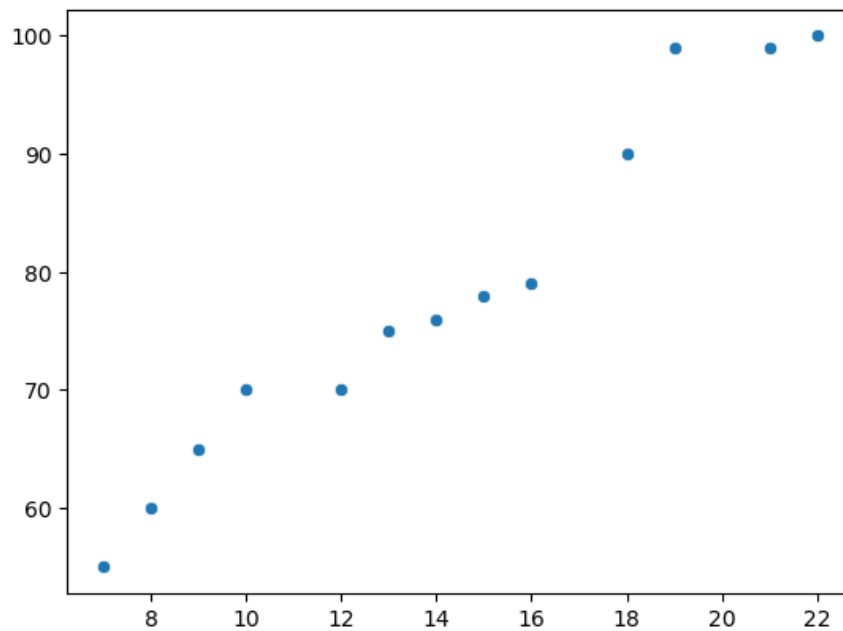
De fet si fem la recta de regressió s'ajusta, però veiem que hi ha alguns punts que estan per sobre i alguns punts que estan per sota.

En canvi si hi apliquem una corba aquesta sembla que s'ajusta molt millor a les nostres dades.

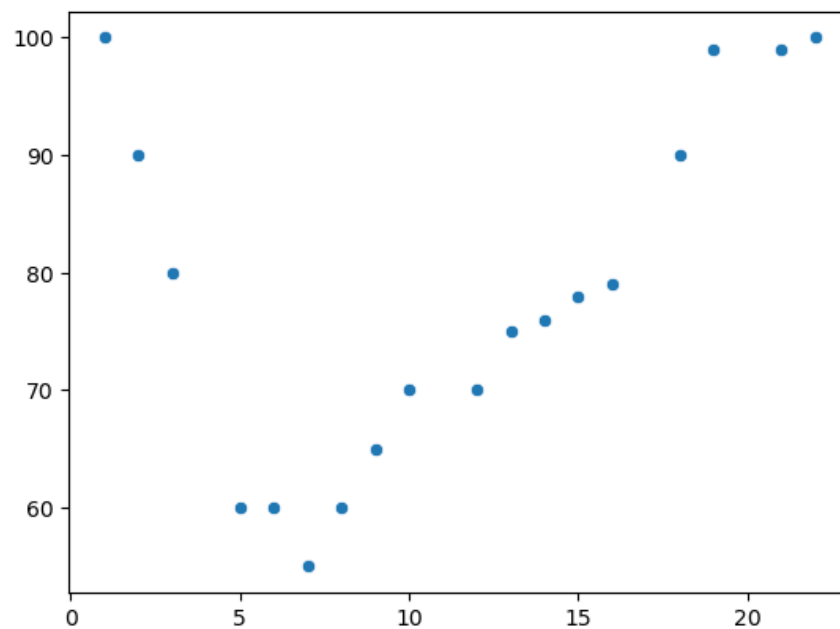


Exemple 2

En aquest segon exemple podem intuir que una recta de regressió lineal simple ens modelarà bé el model

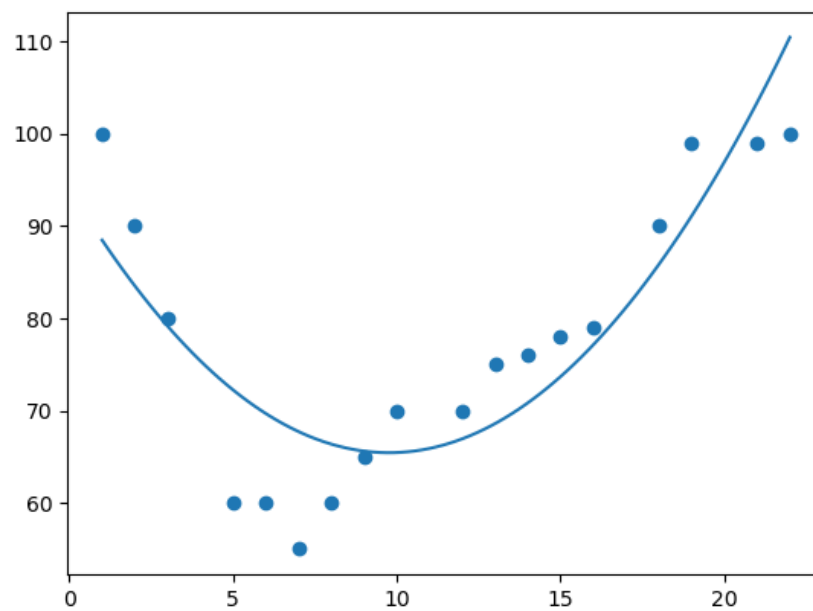


Però resulta que ens donen algunes dades més i veiem el següent mapa de punts.

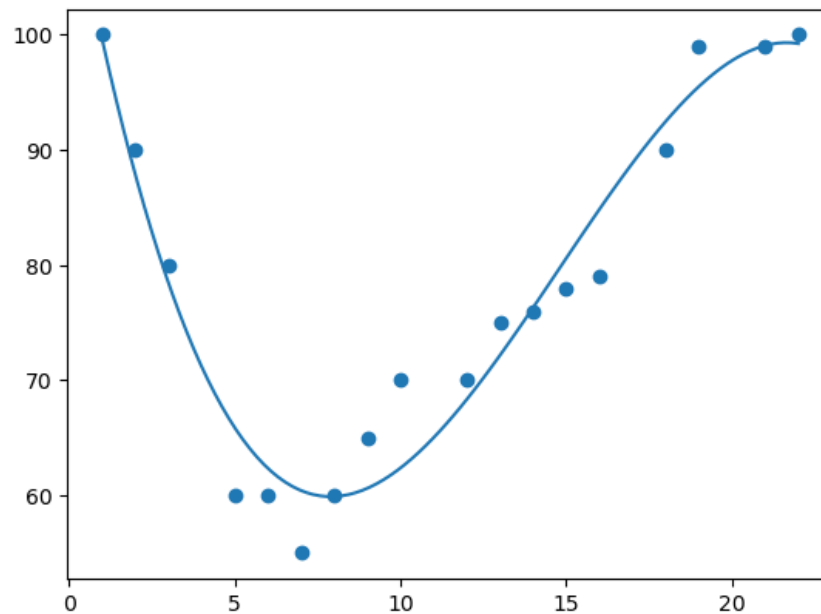


En aquí ja veiem que amb una recta no podrem representar els punts i és aquí a on entraria una regressió lineal polinòmica.

Quan ho provem amb una corba (polinomi de grau 2) tampoc veiem una aproximació al model



Però en canvi, si ho provem amb un polinomi de grau 3 si que veiem que la línia representa força bé les nostres dades.



Avantatges d'ús de la regressió polinòmica

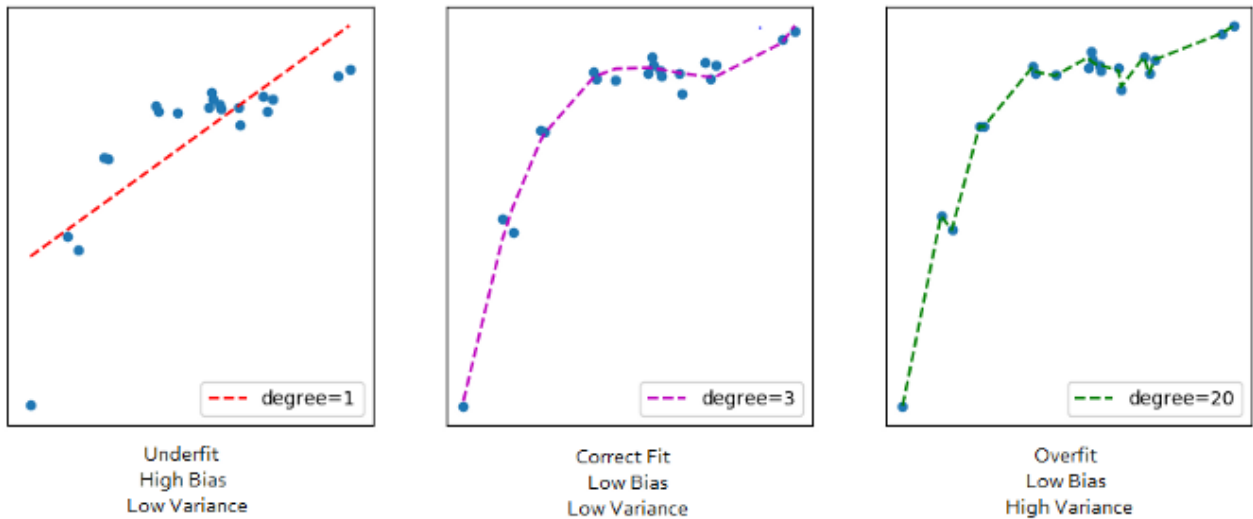
- Com s'ha dit anteriorment podem modelar relacions no lineals entre variables
- Augmentem la varietat de funcions diferents que podem utilitzar per ajustar el model.
- En definitiva, és una eina flexible que es pot utilitzar per adaptar-se a una gran varietat de distribucions de punts de dades.

Problemes de la regressió polinòmica

Encara que podem pensar que la regressió polinòmica ens pot solucionar molts problemes, no és una eina universal i desafortunadament també té un bon nombre de problemes:

- Els models de regressió polinòmica són propensos a sobre ajustar-se. A mesura que augmentem el grau del polinomi per obtenir menys error podem tenir un sobre ajust que ens pot portar a un overfitting ràpidament.
- Hem de focalitzar-nos en els extrems de la corba i veure si les formes i/o tendències tenen sentit.

Perquè polinomis de grau elevat poden produir resultats estranys a l'extrapolació.



Exemples

[0220 regressio lineal polinomica](#) : Exemples de la construcció dels gràfics anteriors

[0221 regressio lineal polinomica - pressio by temperatura](#): Exemple de creació d'un model de regressió polinòmic amb Python i Scikit-learn

Models lineals regularitzats

Quan entrenem un model, especialment un model complex (com una regressió amb moltes variables o una xarxa neuronal amb moltes capes), **el model pot aprendre massa bé les dades d'entrenament provocant sobreajustament (*overfitting*)**.

La **regularització** introdueix un mecanisme que **penalitzava la complexitat del model** per evitar aquest problema.

La regularització consisteix a **afegir un terme de penalització** a la funció de cost o pèrdua (*loss function*) del model.

Si en un model de regressió, per exemple, el que intentem de minimitzar és l'error quadràtic mig MSE (funció de pèrdua L2) a on penalitzem els valors de paràmetres massa grans.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Amb la regularització afegim un terme addicional:

$$MSE_{reg} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot P(\beta)$$

On:

- λ és el paràmetre de regularització (a vegades anomenat *alpha* α), que controla la intensitat de la penalització.
- $P(\beta)$ és la funció de penalització, que pot variar segons el tipus de regularització

Regularització L1 (Lasso Regression)

La regressió Least Absolute Shrinkage and Selection Operator (Lasso) és una versió de regularització que **penalitz**a els valors absoluts dels coeficients:

$$P(\beta) = \sum_{j=1}^n |\beta_j|$$

Això fa que alguns coeficients **es converteixin exactament en zero**, fent que el model elimini variables no rellevants.

Aquesta versió de regularització **ajuda a seleccionar els variables més importants** ja que podríem dir que estira els coeficients petits cap a zero.

Quan la fem servir:

- Tens moltes variables, però sospites que només unes poques són realment útils.
- Necessites fer selecció automàtica de característiques (*feature selection*).
- Vols obtenir un model més simple i interpretable.

Per **exemple** volem predir el rendiment d'un estudiant a partir de 50 variables (hores d'estudi, son, estrès, dietes, ús del mòbil, etc.), però sospites que només 5 o 6 són rellevants.

Compte !!

- Pot eliminar massa variables si α és alt o si hi ha correlació entre variables.

Quina diferència hi pot haver respecte la selecció de variable mitjançant Forward do Backward selection?

Són mètodes estadístics clàssics basats en regressió lineal tradicional (OLS) i criteris estadístics com: p-value dels coeficients, ..

Avantatges dels mètodes de selecció:

- Interpretació clara i clàssica (basada en significació estadística).
- Bona per conjunts petits o models interpretables.
- Ideal per entendre *quines variables realment expliquen Y*

Inconvenients dels mètodes de selecció:

- Pot ser computacionalment lent amb moltes variables.

- **No té en compte la multicolinearietat** → pot deixar variables redundants.
- Pot fer *overfitting* igualment (triar el model que millor s'adapta a l'entrenament però no generalitza).
- Petits canvis a les dades poden canviar el model final.

Per exemple el mètode de *Backward* o *Forward* (p-value) va provant ingredients un a un, tastant la sopa i traient l'ingredient que no aporta sabor significatiu (manual, estadístic, lent). En canvi el Lasso Regressions és com si tinguessis un robot que ajusta automàticament a quantitat de cada ingredient i elimina els que no aporten sabor (automàtic, computacional, modern).

Regularització L2 (Ridge *Regression*)

La regressió Least Absolute Shrinkage and Selection Operator (Lasso) és una versió de regularització que penalitza els quadrats dels coeficients.

$$P(\beta) = \sum_{j=1}^n \beta_j^2$$
$$MSE_{reg} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot P(\beta)$$

Això fa que els coeficients grans es redueixin gradualment, però no arriben a ser exactament zero.

És útil quan totes les variables tenen una mica d'importància.

És útil per reduir la variància del model (quan canvien les prediccions si canviem una mica les dades d'entrenament) o quan tenim moltes variables correlacionades

Direm que el model:

- Té alta variància si el model és molt “sensible” a petites variacions en les dades
- Té baixa variància si el model és estable i canvia poc.

Quan la fem servir:

- Tens **moltes variables** i sospites que **algunes estan correlacionades**.
- **No vols eliminar** variables, només **reduir el seu pes**.
- Vols evitar que alguns coeficients es disparin i provoquin **sobreajustament**.

Per **exemple** volem predir la demanda d'energia d'una ciutat amb moltes variables (temperatura, hora del dia, humitat, dia de la setmana, etc.), on algunes estan correlacionades.

Regularització xarxa elàstica (*Elastic Net*) – L1 +L2

La regularització *Elastic Net* combina L1 i L2

$$P(\beta) = r \sum_{j=1}^n |\beta_j| + (1-r) \sum_{j=1}^n \beta_j^2$$

Aquest mètode de regularització aprofita el millor de cada tipus:

- L1 → redueix a zero alguns coeficients.
- L2 → manté l'estabilitat del model.

En aquest cas el terme de regularització és la suma ponderada dels termes de regularització de Ridge i Lasso i es pot controlar la relació mitjançant r .

- Quan $r = 0$, la xarxa elàstica és equivalent a la regressió Lasso (L1)
- Quan $r = 1$, la xarxa elàstica és equivalent a la regressió Ridge (L2)

Resum

Model	Penalització	Efecte	Elimina variables?	Ús ideal
Linear Regression	✗ Cap	Sense control de complexitat	✗ No	Models simples i interpretables
Ridge (L2)	Quadrats dels coeficients	Redueix coeficients grans	✗ No	Quan hi ha correlacions i moltes variables útils
Lasso (L1)	Valors absoluts dels coeficients	Força coeficients a zero	☑ Sí	Per fer selecció de variables
Elastic Net (L1 + L2)	Combinació	Equilibra L1 i L2	☑ Sí (parcialment)	Quan hi ha moltes variables i correlacions

Imaginem que estàs entrenant un model com si fos un **gimnasta**:

- **LinearRegression:** pot estirar-se tant com vulgui per ajustar-se a cada punt. Risc d'“esquinçar-se”

(sobreajustament).

- **Ridge (L2):** porta una **faixa elàstica** que li limita els moviments extrems. Es mou prou, però amb control.
- **Lasso (L1):** a més de la faixa, té un entrenador que li diu: “Si aquest múscul no serveix, no l’activis” → elimina variables inútils.

Elastic Net: combina les dues estratègies.

Recursos:

- Tutorial “Step-by-Step Guide to Implementing Elastic Net Regression in Python”: https://ujangriswanto08.medium.com/step-by-step-guide-to-implementing-elastic-net-regression-in-python-eff1757aad0a?utm_source=chatgpt.com
- Kaggle Notebook “Multicollinearity / Ridge / Lasso / ElasticNet Regression”: https://www.kaggle.com/code/pranjalpandey12/multicollinearity-ridge-lasso-elasticnet-regression?utm_source=chatgpt.com
- Analytics Vidhya — “Ridge and Lasso Regression in Python” (Tutorial complet amb exemples i interpretació de resultats): https://www.analyticsvidhya.com/blog/2016/01/ridge-lasso-regression-python-complete-tutorial/?utm_source=chatgpt.com
- Car prediction: <https://www.kaggle.com/code/mohaiminul101/car-price-prediction>

Configuració hiperparàmetre

Tots aquests 3 models tenen un hiperparàmetre que controla la força de la regularització, i trobar el millor valor d'aquest hiperparàmetre és clau per obtenir el millor rendiment del model.

Què passa si α és massa gran o massa petit?

- **α Molt petit ($\rightarrow 0$):** Penalització gairebé nul·la $\rightarrow$ model similar a una regressió lineal normal. Pot sobreajustar.
- **α Molt gran:** Penalització molt forta $\rightarrow$ els coeficients s'acosten a 0 $\rightarrow$ el model es torna massa simple (infraajustat).

Com trobem el millor valor d' α ?

Una manera de fer-ho és mitjançant fent una cerca amb validació creuada (*SearchCV*) Scikit-learn té versions dels models **Ridge** i **Lasso** que ja fan la validació creuada internament.

```
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes

# Dades d'exemple
X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Ridge amb validació creuada
ridge_cv = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5)
ridge_cv.fit(X_train, y_train)
print("Millor alpha Ridge:", ridge_cv.alpha_)

# Lasso amb validació creuada
lasso_cv = LassoCV(alphas=[0.001, 0.01, 0.1, 1, 10], cv=5, random_state=42)
lasso_cv.fit(X_train, y_train)
print("Millor alpha Lasso:", lasso_cv.alpha_)
```

alphas $\rightarrow$ conjunt de valors que volem provar.

cv=5 → fa validació creuada de 5 particions.

El model escull l'alpha que dona el menor error mitjà de validació.

Resultat: `ridge_cv.alpha_` i `lasso_cv.alpha_` t'indiquen el millor valor automàticament

Per ElasticNetCV

Funciona igual, però també ajusta el `l1_ratio`, que és la proporció entre L1 i L2.

```
from sklearn.linear_model import ElasticNetCV

elastic_cv = ElasticNetCV(
    alphas=[0.001, 0.01, 0.1, 1, 10],
    l1_ratio=[0.1, 0.5, 0.9],
    cv=5,
    random_state=42
)
elastic_cv.fit(X_train, y_train)

print("Millor alpha:", elastic_cv.alpha_)
print("Millor l1_ratio:", elastic_cv.l1_ratio_)
```

GridSearchCV

En existeix `GridSearchCV` que és capaç de buscar el millor hiperparàmetre de qualsevol model (no només regressions).

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

params = {"alpha": [0.01, 0.1, 1, 10, 100]}
ridge = Ridge()

grid = GridSearchCV(ridge, params, cv=5, scoring="neg_mean_squared_error")
grid.fit(X_train, y_train)

print("Millor alpha:", grid.best_params_["alpha"])
print("Millor score:", -grid.best_score_)
```

Suport Vector Machines
Decision Trees i Random Forest
Adaboost

Models supervisats de classificació

Navie Bayes

K-Nearest Neighbors

Regressió logística

Support Vector Machines

Decision Trees i Random Forest

Adaboost

REFERÈNCIES

- Aurélien Géron (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
- <https://medium.com/@vishalgimhan/types-of-machine-learning-approaches-part-1-3-ml02-0042e443d989>
- Continguts de la docent Cristina Gómez de IES de Teis Vigo